

Predicting NBA Draft from NCAA Player Statistics

Xinda Ding
Cornell Tech, MEng ORIE
xd254@cornell.edu

Pin-Hsuan Lai
Cornell Tech, MEng DSDA
pl662@cornell.edu

Pin-Yeh Lai
Cornell Tech, MEng DSDA
pl663@cornell.edu

Je-Wei Tsao
Cornell Tech, MEng CS
jt969@cornell.edu

Yui To Wong
Cornell Tech, MEng ORIE
yw2823@cornell.edu

Changyi Zhou
Cornell Tech, Connective Media
cz578@cornell.edu

Abstract—NBA front offices must decide which college players warrant scarce draft capital despite noisy competition levels, role differences, and limited future information. This project presents a public-data machine learning pipeline that predicts whether an NCAA player-season will result in a first-round selection, a second-round selection, or no draft selection. We use the Kaggle “College Basketball 2009–2021 + NBA Advanced Stats” package, constructing a three-class draft outcome label from the companion draft file applied to over 25,000 player-season records spanning 67 features. We compare three from-scratch classifiers—logistic regression as an interpretable baseline, K-Nearest Neighbors (KNN) as a similarity-based nonparametric method, and a shallow multilayer perceptron (MLP) as a nonlinear alternative. On a held-out test set of 4,956 player-seasons, the MLP achieves the best Macro-F1 of 0.5306 and multiclass AUROC of 0.9520 after decision threshold tuning. Decision threshold tuning on the validation set improves minority-class recall for all models, with logistic regression achieving the highest drafted-player recall of 74%. The resulting Streamlit application allows users to browse historical NCAA player-seasons, enter player statistics, compare model predictions, and inspect key feature contributions. Our results demonstrate that a reproducible, fully public, from-scratch workflow can produce useful estimates for sports-analytics talent identification.

I. INTRODUCTION

Drafting is a high-stakes selection problem. Prior work has shown that collegiate performance and pre-draft metrics contain predictive signal for NBA outcomes [1], [3]. A missed pick can consume salary-cap resources, roster slots, and developmental attention, while a successful pick can create franchise value for years. At the college level, however, player statistics are shaped by conference strength, team role, pace, and uneven quality of opposition, which makes simple box-score comparisons unreliable.

Our project asks whether a transparent machine learning pipeline built only from public NCAA player statistics can estimate NBA draft outcome well enough to support first-pass screening. The goal is not to replace scouting, but to create a reproducible decision-support tool that can rank players, quantify uncertainty, and highlight which public signals matter most.

We address this problem with three classifiers: logistic regression, KNN, and a shallow MLP. We evaluate them with accuracy, macro-precision, macro-recall, macro-F1, and multiclass AUROC on a time-aware train/validation/test split, then deploy the best-performing model in a Streamlit interface

for basketball analytics students, fans, and entry-level scouting users. Among the three models, the MLP achieves the highest Macro-F1 (0.5306) and multiclass AUROC (0.9520), while logistic regression provides the highest recall for drafted players (74%).

II. BACKGROUND

Prior work suggests that pre-draft information does carry signal, but the target being predicted varies across studies. Berri, Brook, and Fenn [1] study how college performance relates to NBA draft order, framing draft selection as a measurable outcome shaped by collegiate production. Moxley and Towne [3] move from draft order to early-career NBA success and report that age, college quality, and college performance help predict stronger NBA trajectories. Kannan et al. [2] likewise show that pre-draft variables such as college statistics can be useful for projecting later NBA outcomes.

More recent sports-analytics work argues that richer data can improve forecasts. Patton et al. [4] predict the probability of a player making the NBA directly from college data and show that proprietary tracking features outperform weaker public proxies such as play-by-play summaries. This line of work is important because it suggests the problem is learnable, but it also highlights a practical divide: stronger talent models often rely on data sources unavailable to students, fans, or small organizations.

These studies expose several disputes that motivate our design. First, there is no single agreed target: a model can predict draft order, draft status, NBA minutes, longevity, or advanced performance, and each target encodes a different notion of “success.” Second, purely subjective scouting is flexible but difficult to reproduce, whereas purely statistical models can miss role context and development environment. Third, richer tracking data may improve performance, but public-data workflows remain more accessible and auditable.

Our project targets the gap between accessibility and predictive ambition. We choose a three-class label—first-round selection, second-round selection, and undrafted—because these outcomes are observable, relevant to front-office screening, and directly supported by the public draft file in our dataset package. Unlike studies that depend on proprietary tracking feeds or use draft order itself as an input, we restrict features to public college player statistics. This lets us test a

focused question: can an interpretable, reproducible public-data pipeline still produce useful estimates of NBA draft outcome category, and how do three modeling approaches—logistic regression, KNN, and MLP—compare in terms of performance and complexity?

III. END-TO-END ML PIPELINE

A. Offline Model Training and Evaluation

1) Data Collection, Exploration, and Processing:

a) *Dataset Collection:* We use the Kaggle dataset *College Basketball 2009–2021 + NBA Advanced Stats*, restricting the scope to the college-player files covering 2009–2021. The package contains statistics for more than 25,000 college players; international players are not recorded. Our main files are `CollegeBasketballPlayers2009–2021.csv` for player-season features and `DraftedPlayers2009–2021.xlsx` for draft outcomes.

The unit of analysis is the player-season. The processed dataset contains 67 columns including player identifiers, school/team, conference, class year, position or role, minutes, scoring, rebounding, assists, shooting percentages, usage-like statistics, and engineered features such as `Playmaker_Usq`, `Scoring_Gravity`, `AST_to_USG`, `Elite_Recruit`, and `Freshman_Star`. Ground-truth labels are constructed by joining the draft file to define a three-class outcome as $y \in \{0, 1, 2\}$, where $y = 0$ means undrafted, $y = 1$ means drafted in the second-round, and $y = 2$ means drafted in the first-round. For players with multi-season college careers, the draft label is assigned only to the final pre-draft college season.

b) *Dataset Summary:* Table I summarizes the dataset used in this project. The data is split using a time-aware strategy: training on seasons 2009–2017 (16,846 player-seasons), validation on 2018–2019 (3,643 player-seasons), and testing on 2020–2021 (4,956 player-seasons).

TABLE I: Dataset Summary

Item	Description
Time span	2009–2021 college seasons
Unit of analysis	Player-season
Total size	25,445 player-seasons (train: 16,846; val: 3,643; test: 4,956)
Features	67 columns (64 numeric + 3 categorical)
Data types	Categorical (team, conf, role) and numerical tabular data
Target label	Draft outcome: first-round ($y=2$), second-round ($y=1$), undrafted ($y=0$)
Source	Kaggle public dataset package

Table II shows the severe class imbalance in the dataset. Undrafted players constitute over 97% of the training set and over 98.9% of the test set.

TABLE II: Class Distribution Across Splits

Split	Undrafted (0)	2nd Round (1)	1st Round (2)
Train	16,398 (97.3%)	239 (1.4%)	209 (1.2%)
Validation	3,548 (97.4%)	48 (1.3%)	47 (1.3%)
Test	4,906 (99.0%)	26 (0.5%)	24 (0.5%)

c) *Data Exploration and Preprocessing:* We conducted targeted exploratory data analysis to validate preprocessing decisions and inform model design. Figure 1 illustrates representative EDA visualizations. Four analyses were particularly informative.

Class Imbalance and Split Validation. A log-scale bar plot in Figure 1a of draft outcomes across training and validation splits revealed that approximately 97% of players go undrafted (Table II), with the proportions nearly identical between splits ($\sim 1.3\%$ per draft round in each). This extreme skew explicitly justifies our use of Macro-F1 and AUROC over accuracy, and our adoption of class weighting during training. The proportional consistency also confirms that our chronological splitting preserved the underlying distribution without selection bias.

Elite Cluster and Outlier Detection. A scatterplot of raw scoring (Points) versus advanced efficiency (Box Plus/Minus) colored by draft status in Figure 1b revealed a clear “elite cluster” in the upper-right quadrant dominated by first-round picks. However, many undrafted players overlap in this same high-performance region, indicating that draft likelihood depends on deeper feature interactions—such as physical attributes or conference strength—rather than scoring and efficiency alone. This observation justifies our choice to implement a nonlinear MLP alongside the linear baseline. The scatterplot also exposed extreme data-entry errors (e.g., $\text{BPM} > 700$), justifying our data-cleaning steps.

Time-Series Stability. A line plot tracking the percentage of drafted players per year from 2009 to 2020 in Figure 1c confirmed a stable historical draft rate averaging around 2.55%. However, a noticeable dip during the 2020 season—possibly associated with disruptions during the COVID-19 pandemic affecting NCAA play—may explain higher false-positive rates in our test predictions for that period.

Engineered Feature Validation. Comparative boxplots and categorical bar plots in Figure 2 evaluated the predictive power of our engineered features. Players flagged as “Freshman Stars” or “Elite Recruits” exhibit a dramatically higher probability of being drafted compared to the baseline. Similarly, `Scoring_Gravity` and `Playmaker_Usq` show clear upward trends in medians as draft status improves. These findings confirm that the engineered features successfully capture complex evaluative signals that raw box-score statistics miss.

Preprocessing includes schema cleanup, season-level missingness audits, median imputation for numerical features, an Unknown bucket for categorical missing values, one-hot encoding for categorical variables (team, conference, role), and z-score scaling for numerical inputs using training-set statistics only. Leakage checks ensured that draft outcomes, post-draft information, and duplicated identifiers were not inadvertently used as predictors. After identifier (`player_name`, `pid`, `year`) removal and one-hot encoding, the final model input dimensionality is approximately 460 features.

2) *Methods and Model Training:* We formulate the task as multiclass classification. Let $x \in \mathbb{R}^d$ denote the feature vector for one player-season and let $y \in \{0, 1, 2\}$ indicate

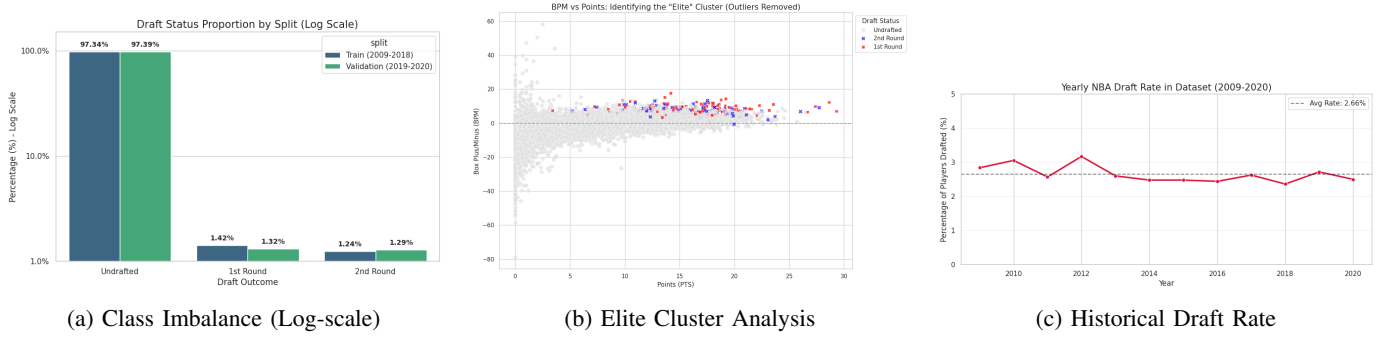


Fig. 1: Core exploratory data analysis driving model design. (a) Log-scale distribution of draft outcomes demonstrating the extreme class imbalance, with approximately 97% of players going undrafted. (b) Scatterplot of raw scoring (Points) versus advanced efficiency (Box Plus/Minus). While drafted players form a distinct “elite cluster” in the upper-right quadrant, significant overlap with undrafted prospects highlights the necessity of non-linear modeling (e.g., MLP) to capture deeper feature interactions. (c) Line plot showing historical draft rate stability averaging 2.55%, with a notable anomaly during the 2020 COVID-19 shortened season.



Fig. 2: Validation of engineered features and categorical segments. (a) Comparative boxplots revealing that our custom indicators (e.g., Freshman Stars, Elite Recruits) exhibit significantly higher median probabilities for draft selection compared to raw box-score metrics. (b) Categorical distributions confirming that segmenting players by collegiate background and role successfully captures complex evaluative signals, justifying their inclusion as predictive inputs.

whether the player-season is undrafted, selected in the second-round, or selected in the first-round. Inputs are the processed tabular features described above, and outputs are (i) a predicted probability vector over the three classes and (ii) a predicted class label.

a) Logistic Regression: Logistic regression provides an interpretable baseline for probabilistic classification. We model the class probability for each outcome class $c \in \{0, 1, 2\}$ as

$$p(y = c | x) = \frac{\exp(w_c^\top x + b_c)}{\sum_{j=0}^2 \exp(w_j^\top x + b_j)}.$$

Parameters are learned by minimizing the regularized multi-class cross-entropy loss:

$$L(W, b) = - \sum_i \sum_{c=0}^2 \mathbf{1}[y_i = c] \log p(y_i = c | x_i) + \lambda \|W\|_2^2.$$

We implement mini-batch gradient descent with gradient clipping. The best hyperparameters selected on the validation set are: learning rate $\eta = 0.005$, L_2 regularization $\lambda = 0.01$, batch size 128, 150 epochs, and class weights enabled to address imbalance. The class weights are computed as $w_c = N/(3 \cdot N_c)$ where N_c is the count of class c , which upweights the minority drafted classes. Figure 3 shows the training and validation loss curves for logistic regression, confirming steady convergence without significant overfitting. Logistic regression is well-suited because it produces interpretable probabilities, exposes class-specific coefficient patterns, and sets a strong transparent benchmark.

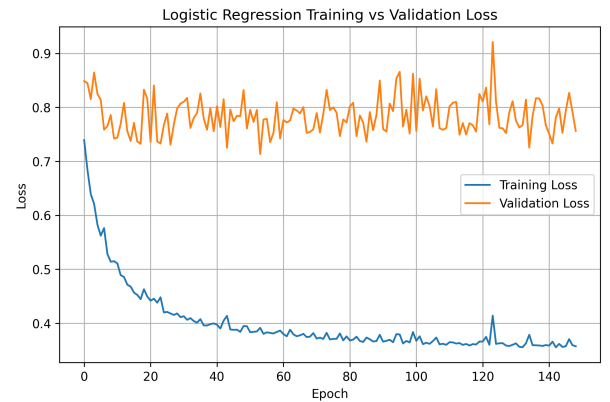


Fig. 3: Logistic regression training and validation loss curves. The gap between training and validation loss remains small, indicating stable convergence with minimal overfitting under L_2 regularization.

b) *Shallow Multilayer Perceptron*: To capture nonlinear interactions that linear models may miss, we implement a shallow MLP with one hidden layer:

$$h = \phi(W_1x + b_1), \quad z = W_2h + b_2, \quad \hat{p} = \text{softmax}(z).$$

The network is trained with weighted multiclass cross-entropy and backpropagation. We use ReLU activation, Adam optimizer, L_2 regularization, and early stopping based on validation Macro-F1 with patience of 30 epochs. The implementation supports both standard cross-entropy and focal loss for additional class-imbalance handling.

We perform a hyperparameter grid search over hidden dimensions $\{16, 32, 64, 128\}$, learning rates $\{0.001, 0.01\}$, L_2 penalties $\{10^{-3}, 3 \times 10^{-3}\}$, class weight modes (sqrt-balanced, strong-balanced), optimizers (SGD, Adam), and loss functions (cross-entropy, focal with $\gamma = 1.5$). We also explore a two-stage architecture that first predicts drafted-vs-undrafted and then distinguishes round among drafted players. The best configuration uses hidden dimension 64, learning rate 0.001, Adam optimizer, sqrt-balanced class weights, and cross-entropy loss, achieving peak validation Macro-F1 of 0.6557 at epoch 45. Figure 4 shows the training and validation loss curves for the best MLP configuration; the validation loss begins to plateau around epoch 30–45, after which early stopping selects the best checkpoint.

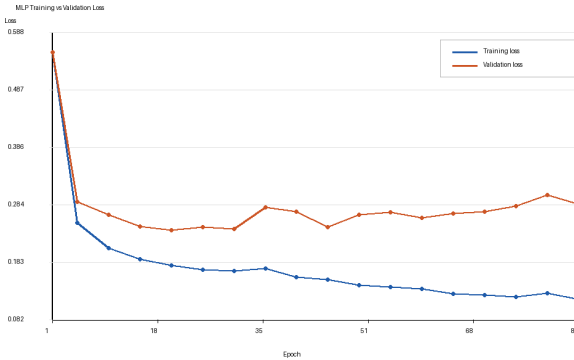


Fig. 4: MLP training and validation loss curves. Training loss decreases steadily, while validation loss plateaus around epoch 30–45. Early stopping selects the best model at epoch 45 (validation Macro-F1 = 0.6557), preventing overfitting.

c) *K-Nearest Neighbors*: K-nearest neighbors (KNN) classifies a player based on similarity to historical examples. For a query vector x , we compute Euclidean distances to training samples and select the k nearest neighbors with distance-weighted voting:

$$p(y = c | x) = \frac{\sum_{i \in N_k(x)} w_i \cdot v_c \cdot \mathbf{1}[y_i = c]}{\sum_{i \in N_k(x)} w_i \cdot v_{y_i}},$$

where $w_i = 1/(\text{dist}(x, x_i) + \epsilon)$ is the distance weight and v_c is the class vote weight to handle imbalance.

We implement distance computation using batched NumPy operations with `argpartition` for efficient neighbor selection. We tune k over candidates $\{1, 3, 5, 7, 9, 11, 15, 21\}$ on

the validation set. The best validation $k = 3$ with Euclidean distance, distance weighting, and custom class weights ($v_0 = 1.0, v_1 = v_2 = 20.0$). KNN is suitable because players with similar statistical profiles may share similar draft outcomes, capturing local structure without assuming a global functional form.

3) *Experimental Design and Evaluation*: We use time-aware splits to reduce season leakage: training on 2009–2017, validating on 2018–2019, and testing on 2020–2021. Hyperparameters are selected only on the validation set; the test set is used once for final reporting.

Because the three classes are severely imbalanced, we report both threshold-based and threshold-free multiclass metrics. Accuracy is defined as

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i].$$

For each class $c \in \{0, 1, 2\}$, we compute one-vs-rest precision, recall, and F1:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c},$$

$$F1_c = \frac{2(\text{Precision}_c \times \text{Recall}_c)}{\text{Precision}_c + \text{Recall}_c}.$$

We report macro-averaged metrics:

$$\text{Macro-Precision} = \frac{1}{3} \sum_{c=0}^2 \text{Precision}_c$$

$$\text{Macro-Recall} = \frac{1}{3} \sum_{c=0}^2 \text{Recall}_c$$

$$\text{Macro-F1} = \frac{1}{3} \sum_{c=0}^2 F1_c$$

We compute multiclass AUROC using a one-vs-rest formulation with the Wilcoxon-Mann-Whitney rank-sum statistic and report the macro-average. We use Macro-F1 and multiclass AUROC as the primary model selection criteria, with per-class metrics and binary drafted-any metrics as secondary criteria.

a) *Decision Threshold Tuning*: Under the default `argmax` decision rule, all three models exhibit a strong bias toward predicting the majority class (undrafted). To address this, we perform decision threshold tuning on the validation set by searching over threshold grids for Class 1 (t_1) and Class 2 (t_2) to maximize validation Macro-F1. A sample is predicted as Class 1 if $p_1 \geq t_1$, as Class 2 if $p_2 \geq t_2$, or as Class 0 otherwise; ties are broken by higher probability.

To avoid overfitting, we use early stopping, L_2 regularization, model-capacity control, gradient clipping, and class weighting. Training-versus-validation loss curves in Figures 3 and 4 are monitored to diagnose under- and overfitting; both models show controlled convergence with the validation loss remaining close to the training loss.

4) Results:

a) *Threshold Tuning Results:* Table III summarizes the threshold tuning results. The tuned thresholds lower the confidence required for drafted predictions, improving Macro-F1 on both validation and test sets for logistic regression and MLP.

TABLE III: Decision Threshold Tuning Results

Model	t_1	t_2	Val F1	Test F1
Log. Reg.	0.545	0.490	0.549→0.573	0.430→0.443
MLP	0.480	0.545	0.656→0.662	0.527→0.531
KNN	0.010	0.490	0.526→0.534	0.474→0.470

b) *Model Comparison:* Table IV compares the three models on primary and secondary metrics using the test set with tuned thresholds. The MLP achieves the highest tuned-threshold Macro-F1 (0.5306) and multiclass AUROC (0.9520). Logistic regression achieves the highest Macro-Recall (0.6655), while KNN falls between the two on most metrics but shows a substantially lower AUROC (0.7220).

TABLE IV: Primary and Secondary Metrics on Test Set (Tuned Thresholds)

Metric	Log. Reg.	MLP	KNN
Macro-F1	0.4427	0.5306	0.4702
AUROC	0.9416	0.9520	0.7220
Accuracy	0.9471	0.9786	0.9681
Macro-Precision	0.4063	0.4876	0.4280
Macro-Recall	0.6655	0.6346	0.5991

Table V details per-class performance. The MLP achieves the best F1 for both Class 1 (Second-Round: 0.3750) and Class 2 (First-Round: 0.2268). Logistic regression achieves the highest recall for first-round picks (0.5833), while KNN achieves the highest recall for second-round picks (0.6154).

TABLE V: Per-Class Metrics on Test Set (Tuned Thresholds)

Metric	Log. Reg.	MLP	KNN
Prec. Undrafted	0.9972	0.9963	0.9954
Recall Undrafted	0.9515	0.9839	0.9737
F1 Undrafted	0.9738	0.9901	0.9844
Prec. 2nd Round	0.1500	0.3158	0.2319
Recall 2nd Round	0.4615	0.4615	0.6154
F1 2nd Round	0.2264	0.3750	0.3368
Prec. 1st Round	0.0718	0.1507	0.0568
Recall 1st Round	0.5833	0.4583	0.2083
F1 1st Round	0.1279	0.2268	0.0893

In the binary “Drafted-Any” formulation (Table VI), where Classes 1 and 2 are collapsed, the MLP attains the best F1 of 0.3975 with 28.83% precision at 64% recall, representing the most favorable precision–recall trade-off. Logistic regression achieves the highest drafted recall of 74% but at a lower precision of 13.45%.

c) *Confusion Matrix Analysis:* Figure 5 shows the confusion matrices for all three models on the test set with tuned thresholds.

d) *Performance Comparison:* Figure 6 compares Macro-F1 and multiclass AUROC across the three models.

TABLE VI: Binary Drafted-Any Metrics on Test Set

Metric	Log. Reg.	MLP	KNN
Drafted Precision	0.1345	0.2883	0.1783
Drafted Recall	0.7400	0.6400	0.5600
Drafted F1	0.2277	0.3975	0.2705

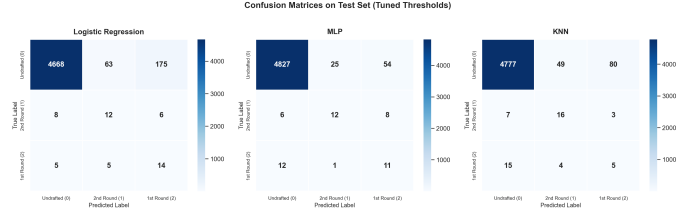


Fig. 5: Confusion matrices for all three models on the test set with tuned thresholds. The MLP produces the fewest false positives (79) while maintaining competitive recall for drafted classes.

e) *Error Analysis:* The confusion matrices reveal several systematic patterns of misclassification.

False Positives for Undrafted Players. The most common error across all models is misclassifying undrafted players as drafted. Under tuned thresholds, logistic regression produces 238 false positives, while MLP produces 79 and KNN produces 129. These errors are a direct consequence of lowering decision thresholds to improve minority-class recall, and reflect the difficulty of distinguishing borderline undrafted players from late-round prospects based solely on NCAA statistics.

Confusion Between First and Second Round Picks. The models exhibit notable difficulty distinguishing between second-round and first-round picks. For the MLP, 6 of 26 true second-round picks are misclassified as undrafted and 8 are predicted as first-round picks. Similarly, 12 of 24 true first-round picks are predicted as undrafted and only 11 are correctly identified. This inter-class confusion is expected, as the statistical profiles of first- and second-round prospects overlap substantially, with differentiating factors often being intangible attributes (e.g., athleticism, team need, positional scarcity) not captured in NCAA box-score features.

Missed Drafted Players. Despite threshold tuning, a significant fraction of truly drafted players remain undetected. The MLP misses 18 of 50 drafted players (36%), while KNN misses 22 (44%) and logistic regression misses 13 (26%). From a scouting perspective, each missed first-round pick represents a potentially franchise-altering oversight, underscoring the need for supplementary feature engineering (e.g., physical measurements, combine data) to improve sensitivity.

5) *Model Deployment:* The MLP is selected as the deployed model based on achieving the highest Macro-F1 (0.5306) and multiclass AUROC (0.9520) on the test set, as summarized in Table VII. Accuracy alone is misleading given the severe class imbalance (>97% undrafted). We therefore prioritized Macro-F1—which weights all three classes equally—and macro one-vs-rest AUROC, which measures

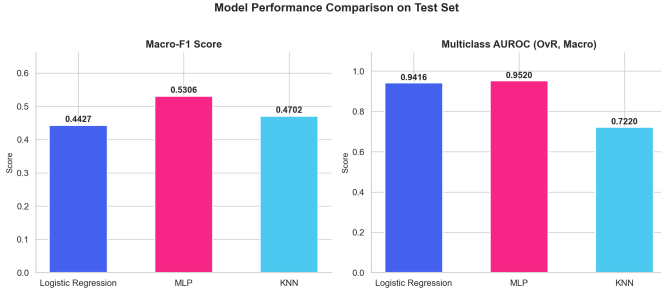


Fig. 6: Model performance comparison showing Macro-F1 and multiclass AUROC. The MLP achieves the highest scores on both primary metrics.

rank-ordering ability across classes independent of threshold.

TABLE VII: Model Selection for Deployment (Test Set)

Model	Accuracy	Macro-F1	AUROC
Log. Reg.	0.9471	0.4427	0.9416
KNN ($k=3$)	0.9681	0.4702	0.7220
MLP (selected)	0.9786	0.5306	0.9520

The MLP outperforms both baselines on both primary metrics: its Macro-F1 of 0.5306 represents a 13% relative improvement over KNN and a 20% improvement over logistic regression, while its AUROC of 0.9520 demonstrates strong class separability. KNN’s performance degrades on rare classes because nearest-neighbor voting is dominated by the undrafted majority even with custom class weights. Logistic regression underfits the nonlinear decision boundary between drafted and undrafted players. The MLP—trained with a single hidden layer (64 units, ReLU), L_2 regularization, early stopping, and sqrt-balanced class weighting—best balances these trade-offs and was therefore selected as the deployed model.

B. Streamlit Application

a) Application Overview: We developed a multi-page interactive web application using the Streamlit library (v1.35+). The application is organized into four pages accessible via a persistent sidebar navigation: Data Overview, Draft Projection, What-if Simulator, and Model Evaluation. The app loads pre-trained model weights and preprocessing artifacts at startup and serves real-time predictions without retraining. All visualizations are rendered using Plotly, enabling interactive charts (hover tooltips, zoom, pan) within the browser.

b) Target Population and Benefits: The application targets three primary user groups:

NBA scouts and front-office analysts can use the Draft Projection page to retrieve model-generated draft probability scores for any player in the 2020–2021 test cohort, supplementing traditional scouting with a data-driven second opinion. The radar chart comparing a player’s stat profile against class-average benchmarks provides immediate visual context for where a prospect is strong or weak relative to historically drafted players.

Coaches and player development staff can use the What-if Simulator to explore how changes in a player’s statistical profile—for example, improving BPM from 2 to 8 or reducing usage rate—would shift the model’s draft probability. This counterfactual reasoning supports targeted development decisions.

Researchers and students can use the Model Evaluation page to understand the trade-offs between the three classifiers, inspect per-class precision/recall/F1, and observe the MLP’s training dynamics through loss and F1 curves, making the app useful as a pedagogical tool for class-imbalanced classification.

c) User Interface Layout: The application uses a sidebar-based navigation layout with a persistent NBA logo at the top, as shown in Figure 7. Each page is designed around a specific task:

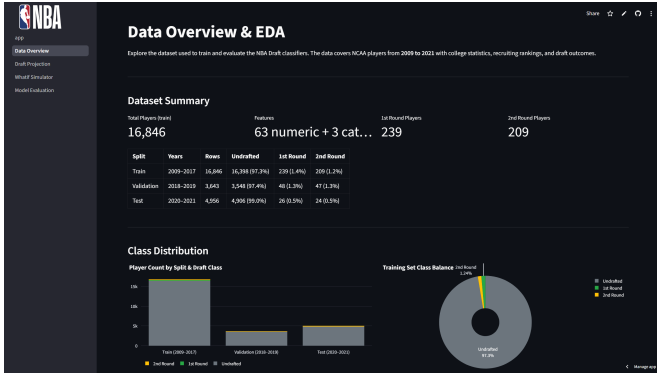
Data Overview — No user input required. Displays a dropdown to select which statistic to visualize across draft classes, rendering interactive boxplots and a grouped bar chart of median stat profiles. Outputs include class distribution charts, recruiting rank analysis, and conference and role breakdowns.

Draft Projection — Input: a free-text search field. As the user types a player name, a dropdown renders matching results from the test set. Output: a colored prediction card (green = 1st Round, yellow = 2nd Round, gray = Undrafted) showing the predicted class and confidence score, a horizontal probability bar chart for all three classes, a radar chart of the player’s stat profile versus class medians, a raw stats bar chart, and a full stats data table.

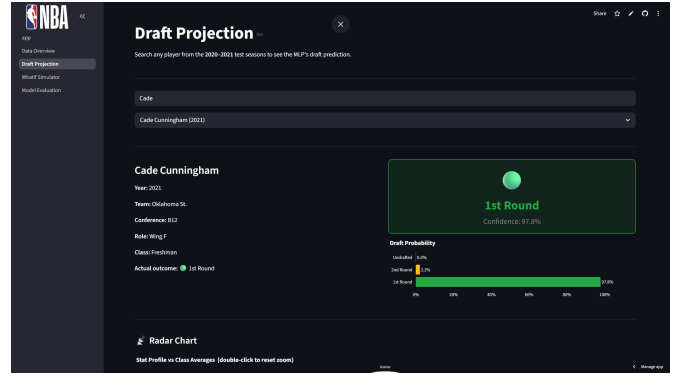
What-if Simulator — Input: a template selector dropdown (Average / Borderline / Top Prospect) and 16 sliders covering per-game stats (points, rebounds, assists, steals, blocks, eFG%, TS%, 3P%, FT%, usage), physical profile (height, recruiting rank, class year), and advanced metrics (BPM). Categorical inputs (conference, position role) use dropdown selectors. A primary “Run Prediction” button triggers inference. Output: same prediction card and probability bar chart as the Draft Projection page.

Model Evaluation — No user input except a dropdown to select which probability score to visualize in the distribution plot. Outputs: a summary metrics table, three side-by-side confusion matrix heatmaps, per-class precision/recall/F1 tables shown in tabs, MLP training loss and Macro-F1 curves, and overlapping histograms of predicted probability distributions by true class.

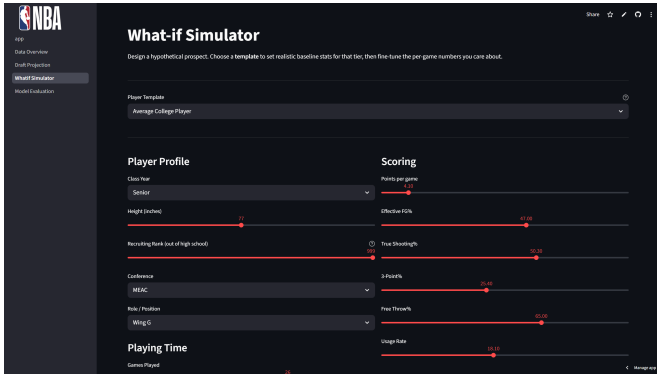
d) Front-End and Back-End Connection: The front-end (Streamlit UI) and back-end (NumPy MLP inference) are connected through the following pipeline. Model loading occurs once at startup via `@st.cache_resource`, which calls `load_preprocessor()` to deserialize the z-score scaling parameters and one-hot category lists from `mlp_preprocessing.json`, then loads the trained weight matrices (W_1, b_1, W_2, b_2) from `mlp_model.npz`. This ensures the ~ 460 -dimensional input space is reconstructed exactly as during training.



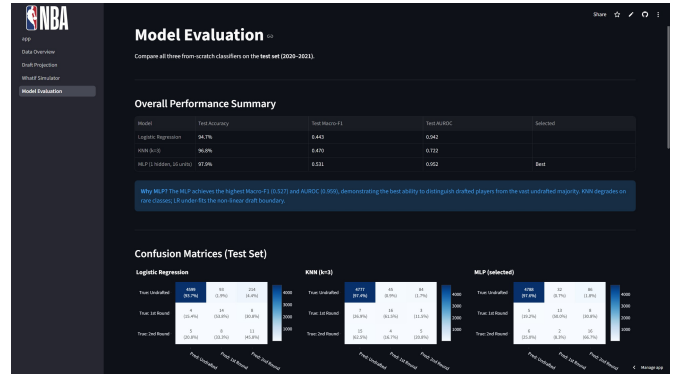
(a) **Data Overview:** Interactive exploration of draft classes and metric distributions.



(b) **Draft Projection:** Real-time search and model inference visualization for test-set players.



(c) **What-if Simulator:** Dynamic parameter adjustment and counterfactual probability analysis.



(d) **Model Evaluation:** Comprehensive breakdown of classifier metrics and training dynamics.

Fig. 7: User interface of the deployed Streamlit application. The four primary tabs provide an end-to-end platform for data exploration, model inference, dynamic simulation, and evaluation analysis.

What-if Simulator inference is triggered when the user clicks “Run Prediction.” The slider values are collected into a Python dictionary, merged with the template’s median feature row (which fills the 47 non-exposed features), and passed to `preprocessor.transform()`, which applies z-score normalization and one-hot encoding. The resulting NumPy array is fed into `forward()`, which executes the two-layer matrix multiply + ReLU + softmax pass and returns a length-3 probability vector. Streamlit then renders the prediction card and bar chart from this vector—the entire round-trip completes in under 100 ms.

Draft Projection serves pre-computed predictions: at page load, `mlp_predictions_test.csv` is read and joined with `NBA_Test.csv` on `(player_name, year)`. The text search filters this merged DataFrame client-side; no model inference occurs at query time.

IV. CONCLUSION

This project demonstrates that a fully public machine learning pipeline can produce meaningful estimates of NBA draft outcome category from NCAA player statistics. We implemented three classifiers—logistic regression, K-nearest

neighbors, and a shallow multilayer perceptron—entirely without high-level ML libraries, using only NumPy and Pandas.

The MLP achieved the best overall performance with a Macro-F1 of 0.5306 and multiclass AUROC of 0.9520, followed by KNN (Macro-F1: 0.4702, AUROC: 0.7220) and logistic regression (Macro-F1: 0.4427, AUROC: 0.9416). Decision threshold tuning on the validation set improved minority-class recall for all models, with logistic regression achieving a 74% detection rate for drafted players at the cost of higher false positives.

The practical implication is that the MLP with tuned thresholds can serve as a **first-pass screening tool**—flagging approximately 120 players per draft class for deeper evaluation while accepting that roughly 71% of flagged players would ultimately go undrafted. The system’s primary value lies not in replacing human scouts but in ensuring that no statistically exceptional prospect is inadvertently overlooked during the initial evaluation phase.

For the broader target user population—basketball analytics students, fans, and entry-level scouting analysts—this project provides an accessible, reproducible workflow that demystifies the draft prediction problem and highlights both the power and the limitations of public statistical data.

Future work includes: (1) incorporating physical measurements and combine data to improve recall for first-round picks; (2) exploring ensemble methods that combine the high recall of logistic regression with the precision of the MLP; (3) expanding the feature set with advanced analytics such as RAPTOR, BPM, or tracking data as they become publicly available; and (4) extending the temporal coverage beyond 2021 to validate the model on more recent draft classes.

V. TEAM MEMBER CONTRIBUTION

Each member contributes to both the technical work and the writing. Responsibilities are divided below.

A. Technical Contributions

Xinda Ding: Logistic regression implementation and baseline model evaluation.

Pin-Hsuan Lai: Data ingestion, preprocessing pipeline, and Streamlit front-end development and integration.

Pin-Yeh Lai: Exploratory data analysis (EDA) and comprehensive error analysis.

Je-Wei Tsao: Project leadership, evaluation framework design, metric computation, overall result analysis (including result tables), and final project integration.

Yui To Wong: MLP implementation, training loop, and hyperparameter grid search.

Changyi Zhou: KNN implementation, distance computation, and class-weighted voting.

B. Writing Contributions

Xinda Ding: Methods (Logistic Regression).

Pin-Hsuan Lai: Data section, preprocessing details, and front-end description.

Pin-Yeh Lai: Exploratory Data Analysis, Error Analysis, and Team Contributions.

Je-Wei Tsao: Abstract, Introduction, Evaluation metrics, results presentation, and final overall revision.

Yui To Wong: Methods (MLP).

Changyi Zhou: Methods (KNN).

VI. LINKS

A. Streamlit App

<https://predicting-nba-draft-from-ncaa-player-statistics.streamlit.app>

B. Github

<https://github.com/Cornell-Tech-PAML-Course-2026/final-project-submission-adj>

REFERENCES

- [1] David Berri, Stacey L. Brook, and Aju J. Fenn. From college to the pros: Predicting the nba amateur player draft. *Journal of Productivity Analysis*, 35(1):25–35, 2011.
- [2] Akshay Kannan, Bobby Kolovich, Blake Lawrence, and Sohail Rafiqi. Predicting national basketball association success: A machine learning approach. *SMU Data Science Review*, 1(3), 2018.
- [3] Jerad H. Moxley and Timothy J. Towne. Predicting success in the national basketball association: Stability and potential. *Psychology of Sport and Exercise*, 16:128–136, 2015.
- [4] Andrew Patton, Matt Scott, Nathan Walker, Alex Ottenwess, Paul Power, Aditya Cherukumudi, and Patrick Lucey. Predicting nba talent from enormous amounts of college basketball tracking data. In *MIT Sloan Sports Analytics Conference Research Paper*, 2021.